QR Image Transfer: File Delivery over a Screen–Camera Link with LT Fountain Codes

A. Eqrash
Independent Researcher

***Abstract*—**We design, implement and measure a system that moves binary data, demonstrated on images up to 3.6 MB, through a one-way optical channel: a laptop browser displays a time-multiplexed stream of QR codes while a phone browser scans, reassembles and verifies the file. From a naive fixed-index loop (224 s for 33.5 kB, stuck at 95%), three diagnosis-driven fixes — integer-scale QR rendering, LT fountain coding, and metadata interleaving — cut the same transfer to 10 s (22x), and delivered a 3,594,667-byte photograph in 1,146 s with SHA-256 match. Every claim is backed by timestamped receiver logs: 7 sessions, over 30,000 decode attempts.

***Index Terms*—**Screen-camera communication, QR codes, fountain codes, LT codes, visible light communication, air-gap data transfer.

I. INTRODUCTION

Moving data between a screen and a camera without radio, cable, pairing or network suits air-gapped machines, exhibition demos and credential-free onboarding: the channel is visible, directional and infrastructure-free. The simplest embodiment — show QR codes as a continuous stream, film it with a phone — is easy to sketch and surprisingly hard to make fast and reliable.

This paper reports a complete build-and-measure cycle. First, a working Web-to-Web system (no installs, no backend) with two protocols: v1 fixed-index chunks and v2 LT fountain drops. Second, a measurement study of 7 instrumented sessions with per-frame arrivals and decoder error histograms. Third, a reproducible diagnosis method: each bottleneck was identified from log statistics — including a 10^{-14} impossibility proof that two frames were systematically unreadable — and fixed with a targeted change, for a 22x speedup on identical hardware.

II. QR CODE PRIMER

A. Structure

The QR code is a matrix symbology standardized in ISO/IEC 18004 [1] (invented at DENSO WAVE [2]). A symbol is a square of black/white *modules*; version V = 1..40 fixes the side length:

$$M = 4V + 17 \quad \text{modules per side}$$

so V1 is 21x21 and V40 is 177x177. Three 7x7 *finder patterns*, separators, alternating *timing patterns* that set the module grid, BCH-protected format info, an alignment grid (V >= 2), and interleaved data/error-correction codewords (Fig. 1) complete the symbol. Physical module pitch is

$$m = W/(4V + 17),$$

with W the displayed width — the most consequential equation in this paper (Section VIII-B).

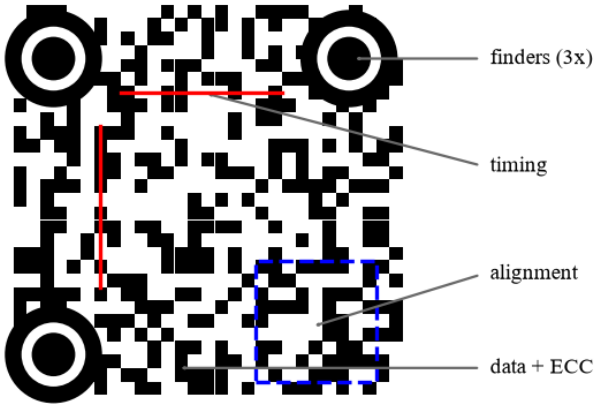


Fig. 1. QR symbol anatomy (schematic, Version 3).

B. Capacity

Capacity depends on version, mode (numeric 3.33 bits/digit, alphanumeric 5.5, byte 8) and ECC level. The V40-L ceilings are 7,089 numeric / 4,296 alphanumeric / **2,953 bytes** / 1,817 Kanji [1], [18]. Byte-mode capacity grows roughly quadratically (Fig. 2).

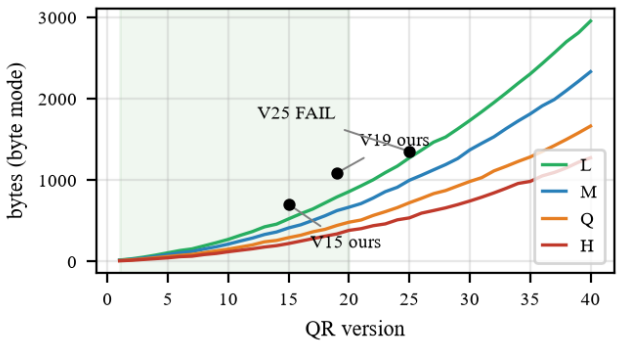


Fig. 2. Byte-mode capacity vs. version and ECC level [1], [18]. Dots: our operating points; V25-Q failed (Sec. VIII-B).

TABLE I

Byte-mode capacities, excerpt (ISO/IEC 18004).

V	L (7%)	M (15%)	Q (25%)	H (30%)
1	17	14	11	7
5	106	84	60	44
10	271	213	151	119
15	523	412	292	220
20	858	666	482	382
25	1,273	997	721	535
30	1,732	1,370	982	742
35	2,303	1,812	1,283	983
40	2,953	2,331	1,663	1,273

Capacity you cannot scan is not capacity [18]: a V40 symbol at 5 cm has 0.27 mm modules, below phone resolution at normal distances. Practical streaming uses mid versions. Raising ECC roughly halves capacity (V40: 2,953 to 1,273 bytes, L to H) from the same codeword budget.

C. Reed-Solomon error correction

Each block protects k data codewords with a Reed-Solomon [20] code over GF(2^8), appending parity that corrects up to t errors:

$$n - k = 2t.$$

Four levels are selectable (Table II); interleaving spreads bursts. Our receiver adds CRC-16 per frame and SHA-256 per file on top.

TABLE II

QR error-correction levels [1].

Level	Recovery	Use here
L	~7%	—
M	~15%	default sweet spot
Q	~25%	tested; V25-Q optically unresolvable
H	~30%	capacity cost too high

III. SCREEN-CAMERA CHANNEL

Streaming QR is screen-camera communication (SCC): surveys report ~900 kbps for streaming codes [8]. Display rate must stay well under half the camera rate or rolling-shutter mixing yields composite frames [10], [11]; perspective, moire, defocus and auto-exposure attack the grid; and phone CPUs decode only a few frames per second in JavaScript, so the sampler — not the display — sets the pace (we measured 14–18 attempts/s). Downscaling to ~640–960 px before decode is standard [17]; camera access mandates HTTPS, shaping our rig.

TABLE III

Prior screen-camera results and this work.

System	Setup	Throughput
Xie et al. [8]	phone-to-phone	~3 kbps
Vanderhulst [8]	loop V1-V40	336 bps-472 kbps
Liu et al. 2020 [9]	dynamic QR	150/300 kbps
Liu colour [9]	3x multiplex	320/900 kbps
Fath et al. [8]	1/2 cam rate	120.43 kbps
FareQR [10]	Viterbi de-mix	block-err fix
RescQR [11]	standard QR	400+ kbps
Chroma/AirCode [12],[13]	imperceptible	~0.55/1 Mbps
This work (v2)	browser, mono	~25 kbps, 3.6 MB file

IV. DATA PREPARATION

Deployed images are already entropy-coded, so generic gzip saves ~0%; the protocol sends raw bytes, with an optional sender-side downscale plus WebP re-encode (typically 5-10x smaller) left to the user so originals stay bit-exact. Text framing adds a 15-byte header plus base64url transport:

$$L_{out} = 4\lceil L_{in}/3 \rceil \quad (+33\% \text{ overhead}),$$

e.g. an 815-byte v1 frame becomes 1,088 characters (about V19-M); a 526-byte v2 drop about 700 (about V15-M). Integrity is layered: Reed-Solomon per symbol, CRC-16 per frame (rejects consistent 1-bit misreads before they poison reassembly), SHA-256 per file. No SUCCESS is declared on hash mismatch.

V. FOUNTAIN CODES

Fixed-index streaming is a coupon-collector process: collecting N specific frames at novel rate lambda takes

$$E[T] = N \cdot H_N/\lambda, \qquad H_N \approx \ln N + \gamma,$$

with the tail dominating (N = 500 at 3/s exceeds 1,000 s). LT codes [4] — the practical digital fountain [5] (see also [6], [7], [21]) — remove *specific* frames: the sender emits endless XOR combinations ('drops') of the K source blocks; the receiver recovers after *any*

$$M = K(1 + \varepsilon)$$

drops via peeling: resolve degree-1 drops, XOR out, cascade. Degrees follow Luby's robust soliton law,

$$\rho(1) = 1/K, \quad \rho(d) = 1/[d(d-1)],$$

$$\tau(d) = R/(dK)\ (d < S), \quad \tau(S) = R\ln(R/\delta)/K,$$

normalized to mu = (rho+tau)/Z (Fig. 3: our K = 66 instance). Degrees derive deterministically from a 32-bit seed, so only (seed, payload) crosses the wire. Our sender transmits K systematic drops first, then endless fresh pseudorandom drops: every received drop helps, whatever came before.

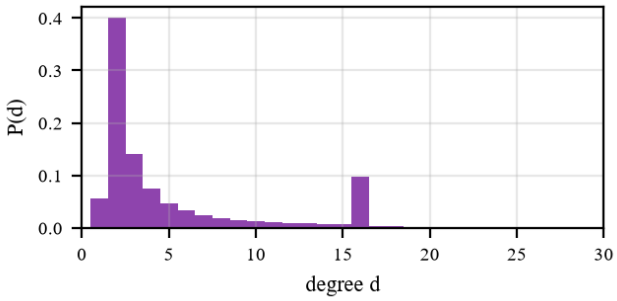


Fig. 3. Robust soliton degree law, K = 66, c = 0.1, delta = 0.5.

VI. SYSTEM DESIGN

Two static pages, no backend, LAN HTTPS. **Sender** (laptop): file to ArrayBuffer, split, QR encode (@zxing/browser [14] decodes; qrcode [19] encodes), fullscreen loop with chained scheduling (each frame fully rendered before the next interval: guaranteed dwell). **Receiver** (phone): getUserMedia at 960x720 (3-5x faster decode), continuous ZXing scan, dedupe map, progress UI, concat, SHA-256 verify, preview. One-tap .txt session reports (per-drop timestamps, 2-s samples, error histogram) back every number in this paper.

TABLE IV

Wire formats (full spec in project docs).

Field	v1 DATA	v2 drop
Magic/ver	'QT'/0x01	'QF'/0x01
Identity	fileId u32	fileId u32
Sizing	total u24	K u24
Position	index u24	seed u32
Check	CRC-16	peeling + SHA-256
Transport	base64url	base64url
META (JSON)	file desc + sha	+ proto/K/blockSize, x10/loop

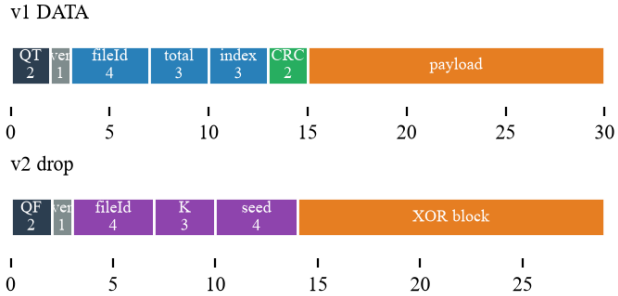


Fig. 4. Frame layouts, widths to scale (payload truncated).

Sessions key off (proto, fileId, total/K) from *any* frame, so mid-loop joins collect immediately; repeat METAs never wipe progress; new fileIds reset cleanly; post-completion repeats are ignored; every v2 pass carries fresh LT seeds, so the stream never repeats a drop.

VII. EXPERIMENTAL METHOD

Hardware: commodity laptop (max brightness, dark surround, fullscreen) to Android 10 / Chrome Mobile 155 at 20-40 cm, perpendicular, office light. **Fixtures:** 33,509-byte PNG (K = 42-66) for iteration; 3,594,667-byte JPEG (K = 4,494 at 800 B) for scaling. **Factors:** block 256-1200 B, fps 4-12, ECC L/M/Q, QR 560/720 px, 2x dwell, v1/v2. **Metrics:** time-to-complete, novel rate, decode ok/attempts, overhead (drops-K)/K, missing set, SHA verdict — all from receiver logs.

VIII. RESULTS

TABLE V

All measured runs (same 33.5 kB image except photo row).

Run	Setup	K-drops	TTC	Valid	Note
v1 base	800B/8fps/M	42/-	224 s	47%	stuck 95%
v2	512B/8fps/M	66-85	82 s	64%	META wait 60 s
v2+META	512B/8fps/M	66-82	18 s	51%	clean
v2 speed	800B/10fps/M	42-47	10 s	47%	12% overhead
v2 photo	800B/10fps/M	4494-5931	1,146 s	53%	3.6 MB ok
v2 dense	1000B/10fps/Q	34-10	stuck	2%	optical wall

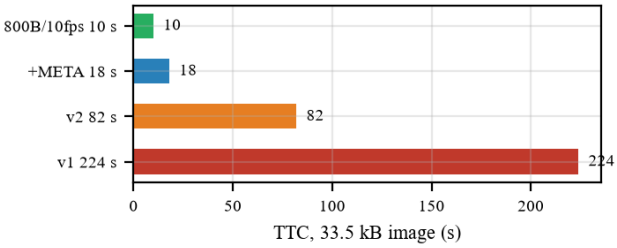


Fig. 5. 224 s to 10 s (22x) on the identical file: one fix per bar.

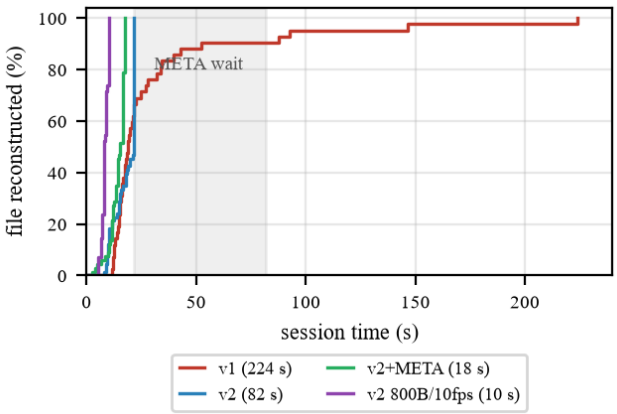


Fig. 6. Collection curves: v1 tail; v2 data in 13 s plus 60 s META wait (shaded); interleaved META removes it.

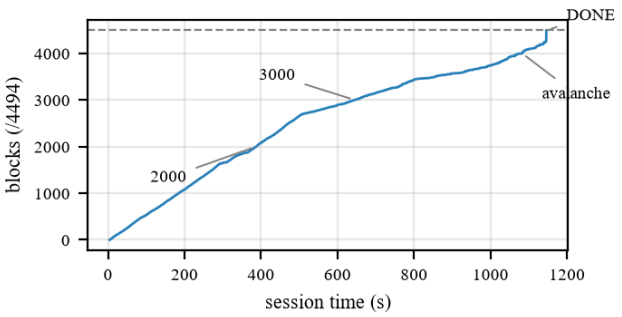


Fig. 7. 3.6 MB photo: linear bulk, grind, then avalanche 4,003 to 4,494 in 64 s.

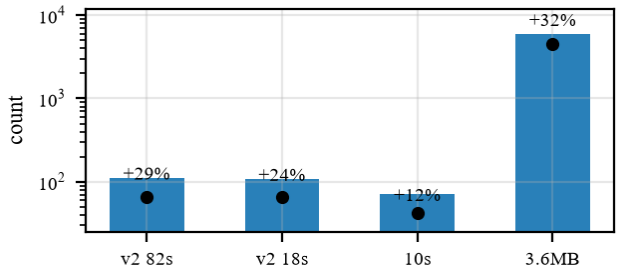


Fig. 8. Fountain overhead: 12-32% from K = 42 to K = 4,494.

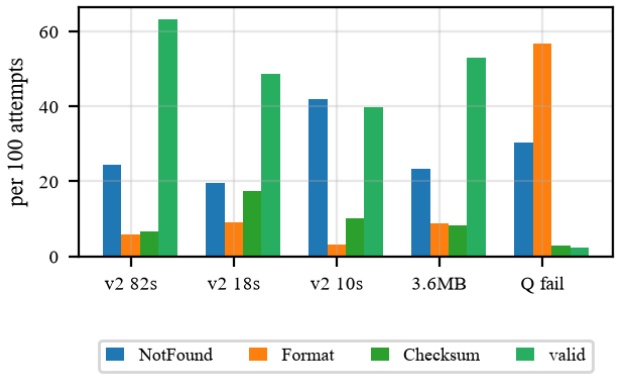


Fig. 9. Decode-error signatures per run (cumulative, per 100 attempts).

A. Four diagnoses, four fixes

(i) Silent SHA crash: Start did nothing on LAN http:// — crypto.subtle is undefined outside secure contexts. Fix: pure-JS fallback plus visible errors. **(ii) Mid-join blindness:** data frames dropped until META; joining a long loop wasted a cycle. Fix: session keyed by any frame. **(iii) Cursed frames:** two indices missing 174 s over ~16 loops at ~40% decode: P = $(0.61)^{64}$ about 10^{-14} under random miss — proof of systematic undecodability. Cause: fixed 720-px canvas forced fractional pixels per module. Fix: integer modules plus pixelated upscale. **(iv) META hostage:** data done in 13 s, DONE 60 s later (META once per 199-frame pass). Fix: META every 20 frames (5% overhead). Next run: 18 s.

Validated goodput model (photo run), with nu valid drops/s, block size B, overhead epsilon:

$$G = \nu \cdot B/(1+\varepsilon) \approx 9.7 \cdot 800/1.32 \approx 3.1 \text{ kB/s},$$

matching 3,594,667 B / 1,146 s. Each loop's per-frame catch probability, explaining the v1 tail:

$$P = 1 - (1 - 1/N)^A,$$

near 1 for small N per loop — yet v1 still stalled, which motivated the 10^{-14} calculation.

IX. LIMITATIONS

Optical wall: 1000 B at ECC Q (about V25) decodes at 2% with FormatException dominant: the sensor cannot resolve the modules hand-held. Ceiling on *this* hardware is 800 B / M (about V19); denser codes need better optics, shorter distance or a stand — not software. **Throughput:** about 3 kB/s net; thermal throttling visibly decays attempt rate after ~3 min. **Security:** integrity without confidentiality — any line-of-sight camera captures the stream; encrypt before encode for sensitive payloads. **Generality:** one phone, one room, engineering (n = 1) runs — effect sizes are large (22x) but absolute times shift with hardware. **UX:** sustained 8-12 fps flashing warrants a photosensitivity warning.

X. FUTURE WORK

In expected return order: (1) fixed mount plus focus lock (attacks the 4.9k NotFounds); (2) RaptorQ precode [7] toward ~5% overhead; (3) colour multiplexing after [9]; (4) adaptive bitrate; (5) encrypted META (X25519 plus AES-GCM); (6) multi-receiver broadcast, the native advantage of a screen over a cable.

XI. CONCLUSION

A QR image stream can carry real files — including multi-megabyte originals — between commodity devices with zero infrastructure, given integer-rendered modules, rateless fountain coding so no frame is special, and metadata frequent enough to never gate completion. Each failure mode (silent crypto, mid-join blindness, cursed patterns, META hostage, optical wall) was diagnosed from log statistics, fixed once, and the 22x speedup held on identical hardware. The remaining limit is photons per second — and the logs say how many.

REFERENCES

[1] ISO/IEC 18004:2024, *Information technology—QR code bar code symbology specification*. Geneva, Switzerland: ISO, 2024. [Online]. Available: iso.org/standard/83389

[2] DENSO WAVE Inc., “Information capacity and versions of QR code.” [Online]. Available: qrcode.com

[3] Thonky, “QR code tutorial.” [Online]. Available: thonky.com

[4] M. Luby, “LT codes,” in *Proc. 43rd Annu. IEEE Symp. Found. Comput. Sci. (FOCS)*, Vancouver, BC, Canada, 2002, pp. 271–280.

[5] J. Byers, M. Luby, M. Mitzenmacher, and A. Rege, “A digital fountain approach to reliable distribution of bulk data,” in *Proc. ACM SIGCOMM*, Vancouver, BC, Canada, 1998, pp. 56–67.

[6] A. Shokrollahi, “Raptor codes,” *IEEE Trans. Inf. Theory*, vol. 52, no. 6, pp. 2551–2567, Jun. 2006.

[7] M. Luby, A. Shokrollahi, M. Watson, and T. Stockhammer, “RaptorQ forward error correction scheme for object delivery,” RFC

6330, Aug. 2012. [Online]. Available: rfc-editor.org

[8] "Streaming QR codes—A survey," *IEEE Xplore*, Art. 10681075, 2024. Consolidates 900 kbps streaming, Xie 3 kbps, Vanderhulst 336 bps–472 kbps, Fath 120.43 kbps. [Online]. Available: ieeexplore.ieee.org

[9] W. Liu *et al.*, "Screen-camera communication system based on dynamic QR code," *IOP Conf. Ser.: Mater. Sci. Eng.*, vol. 790, 2020. 150/300 kbps mono, 320/900 kbps colour. [Online]. Available: iopscience.iop.org

[10] T. Wang *et al.*, "FareQR: fast and reliable screen-camera transfer using QR code," in *Proc. IEEE/ACM SEC*, 2020, pp. 364–369. [Online]. Available: computer.org

[11] "RescQR: enabling reliable data recovery in screen-camera communication," *IEEE Trans. Mobile Comput.*, vol. 23, pp. 3510–3522, May 2024. 400+ kbps. [Online]. Available: computer.org

[12] C. Zhang *et al.*, "ChromaCode: a fully imperceptible screen-camera communication," in *Proc. ACM MobiCom*, 2018. [Online]. Available: purdue.edu

[13] AirCode/ChromaCode slides, USENIX NSDI, 2021 (about 1 Mbps at 5% BER). [Online]. Available: usenix.org

[14] ZXing-JS, "@zxing/browser: BrowserQRCodeReader." [Online]. Available: github.com/zxing-js/browser

[15] "jsQR," npm package (pure-JS ZXing QR port). [Online]. Available: npmjs.com/package/jsqr

[16] "Scan QR codes in browsers with Web Workers and jsQR," How-To Geek. [Online]. Available: howtogeek.com

[17] ZXing-JS issue #236: downscaling high-res inputs fixes detection. [Online]. Available: github.com/zxing-js/library

[18] UseQR, "How much data fits in a QR code." [Online]. Available: useqr.app

[19] "qrcode," npm encoder library. [Online]. Available: npmjs.com/package/qrcode

[20] I. S. Reed and G. Solomon, "Polynomial codes over certain finite fields," *J. Soc. Indust. Appl. Math.*, vol. 8, no. 2, pp. 300–304, Jun. 1960.

[21] D. J. C. MacKay, *Information Theory, Inference, and Learning Algorithms*, Cambridge, U.K.: Cambridge Univ. Press, 2003, ch. 50. [Online]. Available: inference.org.uk

[22] Google, "WebP compression format." [Online]. Available: developers.google.com

[23] National Institute of Standards and Technology, *Secure Hash Standard (SHS)*, FIPS PUB 180-4, Aug. 2015. [Online]. Available: doi.org/10.6028/NIST.FIPS.180-4